

DBMS Lab — Week 2: Queries, Subqueries, Set Operators, and Foreign Key Constraints using a Taxation Database

Aim

To redesign the taxation database created in **Lab-I** by establishing relationships between tables using **Foreign Keys**, and to perform SQL queries using **DISTINCT**, **UNION**, **INTERSECT**, **EXCEPT (MINUS)**, **IN**, **NOT IN**, **EXISTS**, **NOT EXISTS**, **ANY**, and **ALL** operators.

Learning Outcomes

After completing this laboratory, students will be able to:

- Understand the need for relationships between tables.
- Create **Foreign Key** constraints.
- Understand **Referential Integrity**.
- Modify existing tables using **ALTER TABLE**.
- Retrieve data using SQL queries.
- Remove duplicate records using **DISTINCT**.
- Use SQL set operators.
- Write nested queries using subqueries.
- Compare values using **ANY** and **ALL**.
- Understand how relational databases prevent invalid data.

Background

In **Lab-I**, the following tables were created:

Taxpayer
Income_Category
Financial_Year
Income_Record

Each table stored useful information.

However, these tables were **independent**.

For example,

Income_Record

income_id = 1015

taxpayer_id = 999

category_name = Salary

financial_year = 2025-2026

The database accepted this record even if taxpayer **999** did not exist.

Similarly,

category_name = Lottery Income

could also be inserted even if that category was not present in the **Income_Category** table.

This leads to inconsistent and unreliable data.

To solve this problem, relational databases use **Foreign Keys**.

A Foreign Key ensures that data entered into one table must already exist in another table.

Existing Database Structure

The database currently contains:

Taxpayer
Income_Category
Financial_Year
Income_Record

Part A – Redesigning the Database

In Lab-I, the **Income_Record** table stored

category_name
financial_year

as plain text.

This creates unnecessary duplication and allows incorrect values.

In this lab, students will redesign the table by replacing text values with IDs.

Step 1

Modify the **Income_Record** table.

Remove the following columns:

category_name

financial_year

Step 2

Add the following columns:

Column	Data Type
category_id	INT
year_id	INT

Choose appropriate data types.

Step 3

Create Foreign Key constraints.

The relationships should be:

Income_Record.taxpayer_id



Taxpayer.taxpayer_id

Income_Record.category_id



Income_Category.category_id

Income_Record.year_id



Financial_Year.year_id

Students should write the SQL statements required to create these foreign key constraints.

Step 4

Update the existing records in the **Income_Record** table by replacing text values with the appropriate IDs from the corresponding tables.

Part B – Verifying Foreign Keys

Perform the following activities and observe the results.

Task 1

Try inserting a new income record using

taxpayer_id = 999

Observe what happens.

Question

Why does the database reject this record?

Task 2

Try inserting

`category_id = 20`

when only six categories exist.

Observe the error.

Task 3

Try inserting

`year_id = 15`

when that financial year is not available.

Observe the error.

Task 4

Try deleting a taxpayer whose records already exist in **Income_Record**.

Observe the database response.

Task 5

Try deleting an income category that is currently being used by one or more income records.

Observe the response.

Task 6

Write a short note explaining:

- What is a Foreign Key?
 - What is Referential Integrity?
 - Why are Foreign Keys required?
-

Part C – DISTINCT

The **DISTINCT** keyword removes duplicate values from the query result.

Task 1

Display all unique occupations of taxpayers.

Task 2

Display all unique income categories available.

Task 3

Display all unique financial years available in the database.

Task 4

Display all unique income sources recorded in the **Income_Record** table.

Part D – UNION

The **UNION** operator combines the results of two queries and removes duplicate rows.

Task 1

Display the names of taxpayers who have either

- Salary income

OR

- Business income.

Use the **UNION** operator.

Task 2

Display all income sources recorded in

2024–2025

UNION

2025–2026

Task 3

Display taxpayers who belong to the occupation

Teacher

UNION

Software Engineer

Part E – INTERSECT

The **INTERSECT** operator returns only the common rows between two queries.

Task 1

Find taxpayers who have both

- Salary income

and

- Business income.
-

Task 2

Find taxpayers who have income records in both
2024–2025

and
2025–2026.

Part F – EXCEPT (MINUS)

The **EXCEPT** (or **MINUS**) operator returns rows from the first query that are not present in the second query.

Task 1

Find taxpayers who have Salary income but do not have Business income.

Task 2

Find taxpayers who submitted income records in
2025–2026

but not in
2024–2025.

Part G – Nested Queries using IN

The **IN** operator checks whether a value exists in a given set.

Task 1

Display the names of taxpayers who have submitted at least one income record.

Task 2

Display taxpayers whose occupation is present among taxpayers earning Business income.

Part H – Nested Queries using NOT IN

Task 1

Display taxpayers who have not submitted any income records.

Task 2

Display occupations that are not present in any income record.

Part I – EXISTS

The **EXISTS** operator checks whether a subquery returns at least one row.

Task 1

Display taxpayers for whom at least one income record exists.

Task 2

Display financial years that have at least one income record.

Part J – NOT EXISTS

Task 1

Display taxpayers who do not have any income records.

Task 2

Display income categories that have never been used.

Part K – ANY

The **ANY** operator compares a value with at least one value returned by a subquery.

Task 1

Display taxpayers whose annual income is greater than **any** taxpayer whose occupation is **Teacher**.

Task 2

Display taxpayers whose annual income is greater than **any** taxpayer earning Business income.

Part L – ALL

The **ALL** operator compares a value with every value returned by a subquery.

Task 1

Display taxpayers whose annual income is greater than **all** taxpayers whose occupation is **Teacher**.

Task 2

Display taxpayers whose annual income is greater than **all** Business income records.

Part M – Additional Query Practice

Write SQL queries for the following.

1. Display all taxpayers in ascending order of name.
 2. Display taxpayers whose annual income is greater than ₹8,00,000.
 3. Display taxpayers whose occupation is **Software Engineer**.
 4. Display all income records belonging to the **Business** category.
 5. Display income records with amounts between ₹5,00,000 and ₹10,00,000.
 6. Display taxpayers whose names start with the letter **A**.
 7. Display all taxpayers from a particular village or city.
 8. Display all active taxpayers.
 9. Display the total number of taxpayers.
 10. Display the highest annual income recorded.
-

Part N – Mini Challenge

Write SQL queries to answer the following questions.

1. Which taxpayer has the highest annual income?
 2. Which income category contains the highest number of income records?
 3. How many taxpayers belong to each occupation?
 4. How many taxpayers are currently active?
 5. Which financial year contains the highest number of income records?
-

